| | |
|---|---|
| ДИСЦИПЛИНА | Технологии разработки приложений на базе фреймворков |
| ИНСТИТУТ | Институт перспективных технологий и индустриального программирования |
| КАФЕДРА | Индустриального программирования |
| ВИД УЧЕБНОГО МАТЕРИАЛА | Практическая работа |
| ПРЕПОДАВАТЕЛЬ | Ящун Татьяна Викторовна |
| СЕМЕСТР | 5 семестр, 2026-2027 |

# ПРАКТИЧЕСКАЯ РАБОТА № 7
# ОСНОВЫ БАЗ ДАННЫХ И SQL

## ЦЕЛЬ РАБОТЫ

Изучить основные понятия реляционных баз данных и языка SQL, освоить работу с СУБД SQLite: проектирование структуры таблиц, выбор типов данных, определение первичных и внешних ключей, создание таблиц и выполнение запросов на выборку, фильтрацию, сортировку и ограничение данных, применение агрегирующих функций и группировки.

Освоить получение данных из базы SQLite средствами Python и стандартной библиотеки sqlite3, подготовив переход индивидуального проекта от JSON-хранилища к реляционному хранению данных.

## СОДЕРЖАНИЕ


ЦЕЛЬ РАБОТЫ ..... 1
СОДЕРЖАНИЕ ..... 1
1. ЗАДАЧИ РАБОТЫ ..... 3
2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ ..... 3
  2.1. Подготовка ..... 3
  2.2. Выполнение материалов Яндекс Практикума ..... 3
  2.3. Контрольная точка Git ..... 4
  2.4. Создание базы данных индивидуального проекта ..... 4
  2.5. Выполнение SQL-запросов ..... 4
  2.6. Работа с базой данных из Python ..... 5
  2.7. Проверка результата ..... 5
3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ ..... 6
  3.1. База данных и СУБД ..... 6
  3.2. Реляционная модель данных ..... 6
  3.3. Типы данных в SQLite ..... 6
  3.4. Первичный и внешний ключ ..... 7
  3.5. SQL ..... 7

3.6. Основные элементы запроса ....8
3.7. Фильтрация данных ....8
3.8. Сортировка и ограничение выборки ....9
3.9. Агрегирующие функции ....9
3.10. Группировка и фильтрация групп ....9
3.11. Значение NULL ....10
3.12. Работа с SQLite из Python ....10
3.13. Связь с предыдущей реализацией проекта ....11
4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА ....12
5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ ....13
5.1. Общая постановка задания ....13
5.2. Структура базы данных проекта ....14
5.3. Организация файлов ....15
5.4. Создание базы данных и таблиц ....16
5.4.1. Создание файла базы данных ....16
5.4.2. Создание схемы базы данных ....16
5.4.3. Выполнение скрипта во созданию схемы БД ....17
5.4.4. Проверка созданных таблиц ....18
5.5. Перенос данных из JSON в SQLite ....18
5.5.1. Создание подключения к базе данных ....18
5.5.2. Создание скрипта импорта ....19
5.5.3. Выполнение импорта ....21
5.5.4. Проверка импортированных данных ....21
5.6. Разработка SQL-запросов проекта ....23
5.6.1. Подготовка файла SQL-запросов ....24
5.6.2. Запрос 1. Список помещений ....24
5.6.3. Запрос 2. Поиск помещений по вместимости ....25
5.6.4. Запрос 3. Активные бронирования ....25
5.6.5. Запрос 4. Полная информация о бронировании ....26
5.6.6. Запрос 5. Свободные помещения на выбранную дату ....27
5.6.7. Запрос 6. Количество бронирований ....27
5.6.8. Запрос 7. Количество бронирований по помещениям ....28
5.6.9. Запрос 8. Помещения с количеством бронирований больше заданного ....28
5.7. Получение результатов SQL-запросов в Django ....30
5.7.1. Вводная часть ....30
5.7. 1. Настройка получения именованных результатов SQLite ....30
5.7.2. Отображение списка помещений ....31
5.7.3. Отображение активных бронирований ....33
5.7.4. Создание страницы поиска свободных помещений ....34
5.7.5. Создание страницы статистики ....36
5.7. 6. Организация навигации ....37
5.8. Завершение работы ....39
5.8.1 Проверка работы SQL-функциональности ....39
5.8.2. Требования к результату ....40
5.8.3 Обновление README.md ....41
5.8.4 Проверка и фиксация результата ....42
6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ ....43
7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ ....44
7.1. Чек-лист выполнения материалов Яндекс Практикума ....44
7.2. Чек-лист выполнения индивидуального проекта ....45
7.3. Что необходимо показать при защите ....46
7.4. Критерии оценки ....47
7.5. Требования к отчету ....48
7.6. Таблица оценивания индивидуального проекта ....49
8. КОНТРОЛЬНЫЕ ВОПРОСЫ ....51
9. ИТОГ ....53

# 1. ЗАДАЧИ РАБОТЫ

В ходе практической работы необходимо:

- актуализировать знания о реляционных базах данных и СУБД;
- ознакомиться с особенностями SQLite;
- спроектировать структуру базы данных индивидуального проекта;
- определить таблицы, поля и типы данных;
- определить первичные ключи и необходимые внешние ключи;
- создать файл базы данных SQLite;
- создать таблицы средствами SQL;
- перенести существующие данные проекта из JSON-файлов в таблицы базы данных;
- разработать SQL-запросы, реализующие функциональность проекта;
- получать результаты SQL-запросов в Django-представлениях и отображать их на веб-страницах;
- использовать параметризованные запросы при передаче значений;
- сохранить SQL-запросы индивидуального проекта в отдельном файле;
- обновить README.md и зафиксировать изменения в Git.

# 2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ

## 2.1. Подготовка

Работа выполняется в репозитории индивидуального сквозного проекта, созданного в предыдущих практических работах.

- убедитесь, что проект после выполнения ПР6 запускается и содержит результаты предыдущих этапов разработки;
- создайте отдельную ветку или рабочее состояние проекта для выполнения ПР7;
- проверьте наличие Python и возможность создания виртуального окружения;
- установите или проверьте наличие SQLite;
- установите в VS Code расширение для просмотра SQLite-баз данных, если оно ещё не установлено.

**Примечание.** Работа с базой данных в ПР7 выполняется отдельно от Django ORM. На данном этапе используется непосредственно SQLite и библиотека sqlite3 стандартной библиотеки Python.

## 2.2. Выполнение материалов Яндекс Практикума

Выполните материалы раздела 05 «Базы данных и Django ORM» курса Яндекс Практикума «Бэкенд-разработка» в части, предусмотренной для ПР7.

Материалы раздела посвящены основам баз данных: реляционной модели, языку SQL, первым запросам, работе с базой данных из Python, получению данных, фильтрации, сортировке, ограничению выборки, агрегирующим функциям и группировке

записей. Перечень материалов приведён в разделе 4, полный чек-лист для контроля – в подразделе 7.1.

**Примечание.** Материалы раздела, посвящённые Django ORM, моделям Django, миграциям, Django shell и административной панели, выполняются в последующих практических работах.

### 2.3. Контрольная точка Git

Перед началом работы необходимо зафиксировать состояние проекта. Проверить состояние репозитория:

```
git status
```

Если в проекте остались незакоммиченные изменения, сначала завершите работу из ПР6 и сохраните корректное состояние проекта отдельным коммитом:

```
git add .
git commit -m "PR6: завершение работы с шаблонами Django"
```

Каждый коммит, относящийся к ПР7, должен начинаться с метки PR7:. Например:

```
PR7: create SQLite database
PR7: add SQL queries
PR7: add Python database module
```

### 2.4. Создание базы данных индивидуального проекта

На основе предметной области индивидуального проекта выполните проектирование базы данных и подготовьте её к работе:

- определите основные сущности;
- определите состав таблиц;
- определите поля каждой таблицы и их типы;
- определите первичные ключи;
- определите необходимые внешние ключи;
- определите поля, для которых допустимо отсутствие значения;
- создайте файл базы данных SQLite;
- создайте таблицы средствами SQL в файле `sql/schema.sql`;
- создайте модуль подключения к базе данных `db/database.py`.

Структура базы данных должна соответствовать предметной области индивидуального проекта и обеспечивать выполнение основных операций с данными. Пример проектирования и создания базы данных для проекта Roomly приведён в подразделах 5.1–5.4.

### 2.5. Выполнение SQL-запросов

Перенесите существующие данные проекта из JSON-файлов в таблицы базы данных (подраздел 5.5), затем разработайте SQL-запросы, реализующие функциональность проекта (подраздел 5.6). Запросы должны решать задачи предметной области:

- получение списка объектов;
- поиск объектов по заданному критерию (`WHERE`);
- получение связанных данных из нескольких таблиц (`JOIN`);
- подсчёт количества записей и другие агрегатные запросы;
- группировку записей (`GROUP BY`);
- фильтрацию групп (`HAVING`) – при наличии подходящей задачи.

Каждый запрос проверьте в SQLite CLI и сохраните с комментарием в файле `sql/queries.sql`.

Обязательный минимальный объём: не менее 10 `SQL`-запросов, в том числе не менее двух запросов каждого вида (выборка списков объектов, фильтрация `WHERE`, соединение таблиц `JOIN`, агрегатные функции, группировка `GROUP BY`). Запросы, приведённые в подразделах 5.6.2–5.6.9, являются примерами для проекта `Roomly` и служат только образцом, а не руководством к деятельности. Разработанные запросы необходимо перенести в веб-приложение (подраздел 5.7): результаты запросов должны получать `Django`-представления и отображаться на страницах сайта.

### 2.6. Работа с базой данных из Python

Создайте модуль подключения `db/database.py` и скрипт переноса данных `db/import_json.py` (подраздел 5.5). При работе с базой данных из Python:

- подключайтесь к базе данных через `sqlite3.connect()`;
- включайте проверку внешних ключей командой `PRAGMA foreign_keys = ON`;
- выполняйте вставку нескольких записей методом executemany();
- передавайте значения в SQL-запросы через параметры ?;
- сохраняйте изменения командой commit().

В Django-представлениях настройте `conn.row_factory = sqlite3.Row` и обращайтесь к полям результата по именам (подраздел 5.7.2).

**Примечание.** Формирование SQL-запросов с использованием конкатенации строк и f-строк для подстановки пользовательских значений не допускается.

### 2.7. Проверка результата

Проверьте:

- корректность структуры базы данных;
- наличие таблиц;
- перенос данных из JSON-файлов;
- корректность SQL-запросов;
- получение данных в Django-представлениях;
- отображение данных из SQLite на веб-страницах проекта;
- отсутствие ошибок при запуске проекта.

Проверьте проект командами:

```
python manage.py check
pytest -v
flake8 . --extend-exclude=venv
```

Зафиксируйте изменения в Git коммитом с меткой PR7:.

## 3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

### 3.1. База данных и СУБД

База данных – организованное хранилище данных, предназначенное для их сохранения, поиска и обработки.

Система управления базами данных (СУБД) – программное средство, обеспечивающее создание, хранение, изменение и получение данных из базы.

В рамках практической работы используется СУБД SQLite. SQLite хранит базу данных в одном файле. Для учебного индивидуального проекта это позволяет создать полноценную реляционную базу без установки и настройки отдельного сервера СУБД.

### 3.2. Реляционная модель данных

В реляционной базе данных информация представляется в виде таблиц. Основные элементы таблицы:

- таблица – набор данных об однотипных объектах;
- строка (запись) – отдельный объект;
- столбец (поле) – отдельное свойство объекта.

Например, таблица `rooms` может хранить сведения о помещениях:

| id | name | capacity | room_type |
|---|---|---|---|
| 1 | Аудитория 101 | 30 | учебная |
| 2 | Конференц-зал | 80 | конференц-зал |
| 3 | Переговорная 201 | 12 | переговорная |

### 3.3. Типы данных в SQLite

SQLite использует пять основных классов хранения:

- `INTEGER` – целые числа;
- `REAL` – вещественные числа;
- `TEXT` – текстовые данные;
- `BLOB` – двоичные данные;
- `NULL` – отсутствие значения.

При проектировании таблиц необходимо выбирать тип данных в соответствии с назначением поля. Например:

```
CREATE TABLE rooms (
    id INTEGER,
```

```
    name TEXT,
    capacity INTEGER,
    area REAL,
    description TEXT
);
```

### 3.4. Первичный и внешний ключ

Первичный ключ (`PRIMARY KEY`) однозначно идентифицирует запись таблицы. Например:

```
CREATE TABLE rooms (
    id INTEGER PRIMARY KEY,
    name TEXT,
    capacity INTEGER
);
```

Поле `id` позволяет однозначно определить помещение. Внешний ключ (`FOREIGN KEY`) используется для ссылки на запись другой таблицы.

Например, бронирование связано с помещением и пользователем: поля `bookings.room_id` и `bookings.user_id` ссылаются на первичные ключи таблиц `rooms` и `users` (рис. 1).

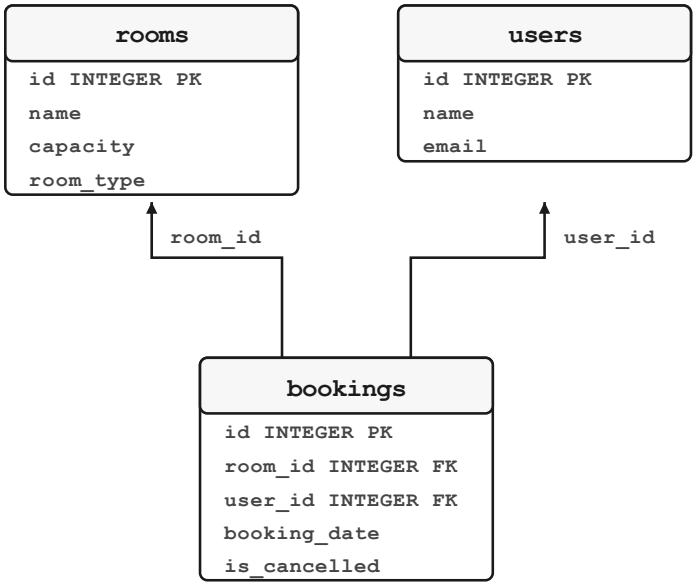



Рисунок 1 – Сущности и связи базы данных проекта Roomly

На этапе ПР7 необходимо спроектировать необходимые ключи, однако подробное изучение связей между таблицами и JOIN выполняется в последующих практических работах.

### 3.5. SQL

SQL (Structured Query Language) – язык для работы с реляционными базами данных. Основная конструкция получения данных:

```
SELECT <столбцы>
FROM <таблица>
WHERE <условие>;
```

Например:

```
SELECT name,
       capacity
FROM rooms
WHERE capacity >= 20;
```

Запрос выбирает название и вместимость помещений, соответствующих заданному условию.

### 3.6. Основные элементы запроса

При выполнении ПР7 используются следующие элементы SQL:

| Элемент | Назначение |
|---|---|
| SELECT | определяет получаемые столбцы |
| FROM | определяет таблицу |
| WHERE | фильтрует записи |
| GROUP BY | объединяет записи в группы |
| HAVING | фильтрует группы |
| ORDER BY | сортирует результат |
| LIMIT | ограничивает количество записей |
| OFFSET | задаёт смещение выборки |

Общий порядок элементов запроса:

```
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
LIMIT
OFFSET
```

Не все элементы являются обязательными.

### 3.7. Фильтрация данных

Для фильтрации записей используется `WHERE`.

Основные операторы:

- =;
- <>;
- !=;
- >;
- <;
- >=;
- <=;
- BETWEEN;
- IN;
- LIKE.

Условия можно объединять с помощью `AND`, `OR` и `NOT`.

Например:

```
SELECT name,
       capacity
FROM rooms
WHERE capacity BETWEEN 10 AND 30
  AND room_type = 'учебная';
```

### 3.8. Сортировка и ограничение выборки

Для сортировки используется `ORDER BY`.

```
SELECT name,
       capacity
FROM rooms
ORDER BY capacity DESC;
```

`ASC` задаёт сортировку по возрастанию, `DESC` – по убыванию.

Для ограничения результата используются `LIMIT` и `OFFSET`:

```
SELECT name,
       capacity
FROM rooms
ORDER BY capacity DESC
LIMIT 5 OFFSET 5;
```

**Примечание.** При использовании `LIMIT` и `OFFSET` для воспроизводимого порядка результатов необходимо задавать `ORDER BY`.

### 3.9. Агрегирующие функции

Агрегирующие функции выполняют вычисления над набором записей. В ПР7 используются:

| Функция | Назначение |
|---|---|
| `COUNT()` | количество |
| `MIN()` | минимальное значение |
| `MAX()` | максимальное значение |
| `AVG()` | среднее значение |
| `SUM()` | сумма |

Пример:

```
SELECT COUNT(*)
FROM rooms;
```

Для `COUNT(column)` значения `NULL` не учитываются.

### 3.10. Группировка и фильтрация групп

`GROUP BY` объединяет записи с одинаковыми значениями указанного поля. Например:

```
SELECT room_type,
       COUNT(*)
FROM rooms
```

```
GROUP BY room_type;
```

Получается количество помещений каждого типа.

HAVING применяется для фильтрации уже сформированных групп:

```
SELECT room_type,
       COUNT(*)
FROM rooms
GROUP BY room_type
HAVING COUNT(*) >= 2;
```

Различие:

- WHERE – фильтрация отдельных записей;
- HAVING – фильтрация групп.

### 3.11. Значение NULL

NULL обозначает отсутствие значения. Для проверки NULL используются IS NULL и IS NOT NULL. Обычное сравнение field = NULL для проверки отсутствующего значения не используется.

Агрегирующие функции COUNT(column), MIN(), MAX(), AVG() и SUM() не учитывают NULL при соответствующих вычислениях.

### 3.12. Работа с SQLite из Python

Для работы с SQLite используется стандартная библиотека Python:

```
import sqlite3
```

Основной порядок работы показан на рис. 2.

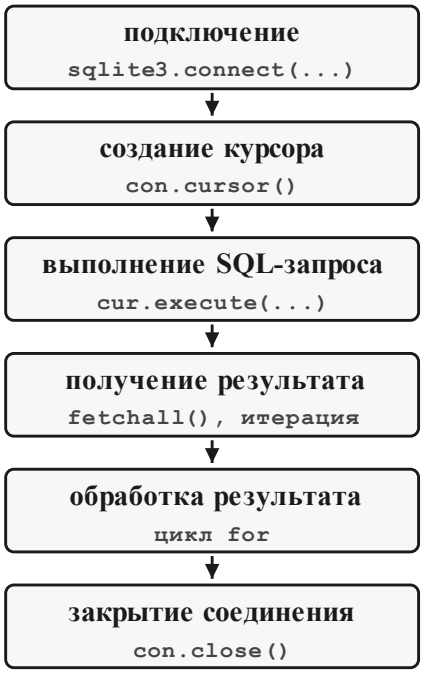



Рисунок 2 – Порядок работы с SQLite из Python

Пример:

```
import sqlite3
con = sqlite3.connect("data/roomly.sqlite")
cur = con.cursor()
results = cur.execute("""
    SELECT name,
           capacity
    FROM rooms
    ORDER BY capacity DESC;
""")
for result in results:
    print(result)
con.close()
```

При выполнении запросов только на получение данных `commit()` не требуется.

Для передачи значений в запрос используются параметры:

```
cur.execute(
    """
    SELECT name,
           capacity
    FROM rooms
    WHERE capacity >= ?;
    """,
    (20,)
)
```

Такой подход позволяет отделить SQL-код от передаваемых значений и избежать небезопасной подстановки данных непосредственно в строку SQL-запроса.

### 3.13. Связь с предыдущей реализацией проекта

До ПР7 данные проекта Roomly хранились в JSON-файлах и обрабатывались средствами Python. В ПР7 появляется отдельный слой базы данных: программа обращается к SQLite через SQL-запросы и модуль sqlite3 (рис. 3).

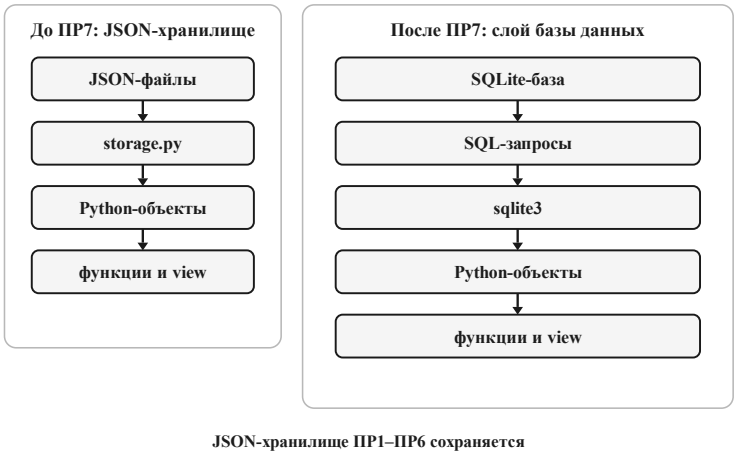



Рисунок 3 – Архитектура данных проекта до и после ПР7

При этом результаты ПР1–ПР6 удалять не требуется: JSON-хранилище сохраняется как предыдущая реализация предметной области.

В ПР7 необходимо сохранить возможность обратиться к исходной реализации проекта и сопоставить операции над данными:

| Предыдущая реализация | ПР7 |
|---|---|
| JSON-файл | SQLite-база |
| словарь/объект Python | запись таблицы |
| поле объекта | столбец |
| список объектов | результирующая выборка |
| поиск в Python | `WHERE` |
| сортировка в Python | `ORDER BY` |
| получение части списка | `LIMIT` и `OFFSET` |
| подсчёт средствами Python | агрегирующие функции SQL |
| группировка средствами Python | `GROUP BY` |

Цель такого сопоставления – определить, какие операции над данными могут быть перенесены из программного кода в запросы к базе данных.

**Примечание.** Код предыдущих практических работ не требуется удалять только из-за появления базы данных. На ПР7 задача состоит в освоении SQL и работы с SQLite, а не в полном переходе проекта на Django ORM.

## 4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА

В рамках ПР7 выполняются материалы раздела 05 «Базы данных и Django ORM» курса Яндекс Практикума «Бэкенд-разработка» темы «Введение в базы данных». Перечень материалов приведён в таблице: материалы отмечаются по мере выполнения. Полный чек-лист для контроля – в подразделе 7.1.

| № | Материал Яндекс Практикума | Выполнить |
|---|---|---|
| 1 | Введение в базы данных / Введение в спринт | ✓ |
| 2 | Введение в базы данных / Базы данных: реляционные и не только | ✓ |
| 3 | Введение в базы данных / SQL: язык структурированных запросов | ✓ |
| 4 | Введение в базы данных / Первые запросы, первые результаты | ✓ |
| 5 | Введение в базы данных / Работа с базой данных из Python | ✓ |
| 6 | Введение в базы данных / Получение данных | ✓ |
| 7 | Введение в базы данных / Практика запросов | ✓ |
| 8 | Введение в базы данных / Сортировка, ограничение и сдвиг выборки | ✓ |
| 9 | Введение в базы данных / Агрегирующие функции | ✓ |
| 10 | Введение в базы данных / Группировка записей в выборке | ✓ |

При выполнении материалов необходимо не только пройти задания Практикума, но и перенести изученные приёмы на базу данных индивидуального проекта.

Материалы раздела, посвящённые Django ORM, в ПР7 не выполняются.

# 5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ

## 5.1. Общая постановка задания

Продолжите разработку индивидуального проекта, созданного в ПР1–ПР6.

На предыдущих этапах данные проекта хранились в JSON-файлах и обрабатывались средствами Python: файлы загружались целиком, данные преобразовывались в объекты Python и передавались в шаблоны Django. Такая схема работоспособна, однако по мере роста проекта усложняется поиск нужных записей, фильтрация, сортировка и согласование данных между собой. Эти операции естественнее выполнять там, где данные хранятся, – средствами базы данных.

В ПР7 индивидуальный проект получает слой реляционного хранения данных. Для проекта Roomly последовательность обработки данных меняется следующим образом. На предыдущих этапах данные проходили путь:

```
JSON → Python → Django → шаблоны → сайт
```

После добавления базы данных данные проходят путь:

```
SQLite
↓
SQL
↓
Python sqlite3
↓
Django views
↓
существующие шаблоны
↓
сайт Roomly
```

В ходе выполнения работы необходимо:

- определить основные сущности предметной области;
- представить каждую сущность в виде отдельной таблицы;
- определить поля таблиц и их типы;
- назначить первичные ключи;
- установить связи между таблицами с помощью внешних ключей;
- создать базу данных SQLite;
- перенести существующие данные из JSON-файлов в созданные таблицы;
- выполнить выборку и обработку данных средствами SQL;
- получать результаты SQL-запросов в Django-представлениях и отображать их на веб-страницах проекта.

Далее в качестве примера рассматривается проект Roomly. Основные сущности этой предметной области:

- помещение;
- пользователь;
- бронирование.

Им соответствуют таблицы:

- rooms;
- users;
- bookings.

Связи между сущностями просты и характерны для многих предметных областей:

- одно помещение может иметь несколько бронирований;
- один пользователь может иметь несколько бронирований;
- каждое бронирование относится к одному помещению и одному пользователю.

Таким образом, таблица bookings содержит внешние ключи room_id и user_id, связывающие её соответственно с таблицами rooms и users.

**Примечание.** Примеры выполнения приведены для проекта Roomly. Их необходимо адаптировать к предметной области собственного проекта: состав сущностей, таблиц, полей и запросов определяется именно вашим проектом. Требования к итоговому результату сформулированы в подразделе 5.8.2 и разделе 6.

## 5.2. Структура базы данных проекта

Определите структуру таблиц собственной предметной области: для каждой таблицы зафиксируйте состав полей, типы данных и ключи. Пример для проекта Roomly.

Таблица rooms:

| Поле | Тип | Назначение |
|---|---|---|
| id | INTEGER | идентификатор, первичный ключ |
| name | TEXT | название помещения |
| capacity | INTEGER | вместимость |
| room_type | TEXT | тип помещения |
| description | TEXT | описание |

Таблица users:

| Поле | Тип | Назначение |
|---|---|---|
| id | INTEGER | идентификатор, первичный ключ |
| name | TEXT | имя пользователя |
| email | TEXT | электронная почта |

Таблица bookings:

| Поле | Тип | Назначение |
|---|---|---|
| id | INTEGER | идентификатор, первичный ключ |
| room_id | INTEGER | ссылка на rooms.id |
| user_id | INTEGER | ссылка на users.id |

| Поле | Тип | Назначение |
|---|---|---|
| booking_date | TEXT | дата бронирования |
| is_cancelled | INTEGER | признак отмены |

Поле id каждой таблицы используется в качестве первичного ключа (PRIMARY KEY). Первичный ключ однозначно идентифицирует запись таблицы и позволяет другим таблицам ссылаться на эту запись.

Поля room_id и user_id таблицы bookings являются внешними ключами (FOREIGN KEY):

- room_id ссылается на rooms.id (таблица rooms);
- user_id ссылается на users.id (таблица users).

Благодаря внешним ключам каждое бронирование связано с конкретным помещением и конкретным пользователем, а база данных отражает связи между сущностями предметной области.

В SQLite логическое значение для поля is_cancelled хранится как число:

- 0 – бронирование активно;
- 1 – бронирование отменено.

Дата booking_date на данном этапе хранится как строка в формате YYYY-MM-DD, например 2026-09-20. Такой формат допускает лексическое сравнение и сортировку дат средствами SQL.

**Примечание.** Структура конкретного проекта может отличаться от примера: состав таблиц и полей определяется предметной областью собственного проекта.

### 5.3. Организация файлов

Добавьте в проект отдельную структуру для работы с базой данных: SQL-файлы схемы и запросов, Python-пакет db/ с модулями подключения и импорта данных, файл базы данных в каталоге data/ – создается далее специальной командой.

```
Roomly/
├── db/
│   ├── __init__.py
│   ├── database.py
│   └── import_json.py
├── sql/
│   ├── queries.sql
│   └── schema.sql
└── data/
    ├── rooms.json
    ├── users.json
    ├── bookings.json
    └── roomly.sqlite  – создается на следующем этапе
```

Назначение файлов:

- sql/schema.sql – SQL-команды создания таблиц;

- sql/queries.sql – SQL-запросы для выборки и обработки данных;
- db/database.py – подключение Python к SQLite;
- db/import_json.py – перенос существующих данных из JSON в SQLite;
- data/roomly.sqlite – файл базы данных SQLite.

Такое размещение отделяет данные от кода и запросов: файл базы данных не смешивается с исходным кодом, а SQL-команды хранятся отдельно от Python-модулей.

Файл db/__init__.py может оставаться пустым. Его наличие позволяет использовать каталог db как Python-пакет и обращаться к его модулям, например

```
from db.database import get_connection.
```

**Примечание.** Названия файлов и каталогов необходимо адаптировать к структуре индивидуального проекта.

## 5.4. Создание базы данных и таблиц

Создайте файл базы данных SQLite и создайте в нём таблицы средствами SQL. Команды данного подраздела выполняются в PowerShell из корневого каталога проекта.

### *5.4.1. Создание файла базы данных*

Если файл базы данных уже существует и её требуется создать заново, удалите старый файл:

```
Remove-Item .\data\roomly.sqlite
```

Создайте пустой файл базы данных средствами Python:

```
python -c "import sqlite3; sqlite3.connect('data/roomly.sqlite').close()"
```

После выполнения команды в каталоге data должен появиться файл roomly.sqlite. Соединение сразу закрывается: на данном этапе достаточно самого файла, таблицы будут созданы отдельными командами.

### *5.4.2. Создание схемы базы данных*

Команды создания таблиц удобно хранить в отдельном файле схемы – при пересоздании базы всю схему можно выполнить заново одной операцией. Откройте файл sql/schema.sql и запишите в него:

```
-- ========================================
-- Roomly: структура базы данных
-- ========================================

CREATE TABLE IF NOT EXISTS rooms (
id INTEGER PRIMARY KEY,
name TEXT NOT NULL,
```

```
capacity INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS users (
id INTEGER PRIMARY KEY,
name TEXT NOT NULL,
email TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS bookings (
id INTEGER PRIMARY KEY,
room_id INTEGER NOT NULL,
user_id INTEGER NOT NULL,
booking_date TEXT NOT NULL,
is_cancelled INTEGER NOT NULL DEFAULT 0,
FOREIGN KEY (room_id) REFERENCES rooms(id),
FOREIGN KEY (user_id) REFERENCES users(id)
);
```

В команде `CREATE TABLE` определяются имя таблицы, состав полей и их типы. Использованные в схеме ограничения:

- `PRIMARY KEY` определяет уникальный идентификатор записи;
- `FOREIGN KEY` определяет связь записи с другой таблицей;
- `NOT NULL` запрещает сохранять в поле значение `NULL`;
- `DEFAULT` задаёт значение, которое используется, если поле при вставке не указано.

Для таблицы `bookings` внешние ключи обеспечивают связи:

```
bookings.room_id → rooms.id
bookings.user_id → users.id
```

Конструкция `IF NOT EXISTS` предотвращает ошибку при повторном выполнении схемы: таблица создаётся только в том случае, если её ещё нет.

### *5.4.3. Выполнение скрипта во созданию схемы БД*

Выполните скрипт средствами `Python`. В `PowerShell`, из корня проекта, выполните команду:

```
python -c "import sqlite3; conn=sqlite3.connect('data/roomly.sqlite');
sql=open('sql/schema.sql', encoding='utf-8').read(); conn.exe-
cutescript(sql); conn.commit(); conn.close()"
```

Метод `executescript()` выполняет несколько `SQL`-команд, разделённых точкой с запятой, поэтому весь файл схемы применяется одной операцией. В результате в базе должны быть созданы три таблицы:

- `rooms`;

- users;
- bookings.

### *5.4.4. Проверка созданных таблиц*

Проверьте состав созданных таблиц. В PowerShell запустите интерактивный Python:

```
python
```

После появления приглашения >>> последовательно выполните каждую из команд:

```
import sqlite3
conn = sqlite3.connect("data/roomly.sqlite")
conn.execute("SELECT name FROM sqlite_master WHERE type='table'").fetch-
all()
```

Ожидаемый результат:

```
[('rooms',), ('users',), ('bookings',)]
```

Проверьте наличие внешних ключей:

```
conn.execute("PRAGMA foreign_key_list(bookings)").fetchall()
```

В результате должны присутствовать связи с таблицами rooms и users. Для выхода из интерактивного режима выполните exit().

Созданную базу данных **рекомендуется** открыть в SQLite Viewer в VS Code и убедиться в наличии таблиц и их полей. На данном этапе SQLite Viewer используется для просмотра структуры базы; выполнение SQL-запросов организуется через SQLite CLI (подраздел 5.6.1).

## 5.5. Перенос данных из JSON в SQLite

В проекте уже существуют данные предметной области – JSON-файлы каталога data: rooms.json, users.json, bookings.json. Перенесите эти данные в созданные таблицы SQLite, чтобы далее обрабатывать их средствами SQL. Для переноса используются два модуля Python: модуль подключения к базе данных и скрипт импорта.

### *5.5.1. Создание подключения к базе данных*

Поместите в файл db/database.py следующее содержимое:

```
import sqlite3

DB_PATH = "data/roomly.sqlite"

def get_connection():
    conn = sqlite3.connect(DB_PATH)
```

```
    conn.execute("PRAGMA foreign_keys = ON")
    return conn
```

Функция `get_connection()` создаёт соединение с базой данных и настраивает его. Команда `PRAGMA foreign_keys = ON` включает проверку внешних ключей для текущего соединения: по умолчанию `SQLite` не контролирует, что значение внешнего ключа ссылается на существующую запись другой таблицы. Включение проверки при каждом подключении гарантирует целостность данных при любых операциях с базой.

### *5.5.2. Создание скрипта импорта*

В файле `db/import_json.py` сформируйте скрипт, который читает `JSON`-файлы проекта и переносит их содержимое в таблицы базы данных:

```
import json
from db.database import get_connection

ROOMS_FILE = "data/rooms.json"
USERS_FILE = "data/users.json"
BOOKINGS_FILE = "data/bookings.json"


def load_json(filename):
    with open(filename, encoding="utf-8") as file:
        return json.load(file)


def import_rooms(conn):
    rooms = load_json(ROOMS_FILE)
    conn.executemany(
        """
    INSERT INTO rooms (
    id,
    name,
    capacity
    )
    VALUES (?, ?, ?)
    """,
        [
            (
                room["id"],
                room["name"],
                room["capacity"],
            )
            for room in rooms
        ],
    )
```

```
def import_users(conn):
    users = load_json(USERS_FILE)
    conn.executemany(
        """
    INSERT INTO users (
    id,
    name,
    email
    )
    VALUES (?, ?, ?)
    """,
        [
            (
                user["id"],
                user["name"],
                user["email"],
            )
            for user in users
        ],
    )

def import_bookings(conn):
    bookings = load_json(BOOKINGS_FILE)
    conn.executemany(
        """
    INSERT INTO bookings (
    id,
    room_id,
    user_id,
    booking_date,
    is_cancelled
    )
    VALUES (?, ?, ?, ?, ?)
    """,
        [
            (
                booking["id"],
                booking["room_id"],
                booking["user_id"],
                booking["booking_date"],
                int(booking["is_cancelled"]),
            )
            for booking in bookings
        ],
    )


def main():
    conn = get_connection()
    try:
```

```
        import_rooms(conn)
        import_users(conn)
        import_bookings(conn)
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

if __name__ == "__main__":
    main()
```

Пояснения к реализации:

- параметры ? используются для передачи значений в SQL-команду без подстановки строк в текст запроса;
- метод executemany() выполняет одну команду INSERT для списка значений, добавляя несколько записей одной операцией;
- int(booking["is_cancelled"]) приводит логическое значение из JSON к целому числу, которое хранится в SQLite;
- commit() сохраняет изменения в базе данных;
- rollback() отменяет все изменения текущей транзакции, если во время импорта возникла ошибка, – база остаётся в исходном состоянии.

### *5.5.3. Выполнение импорта*

Выполните скрипт в PowerShell, из корневого каталога проекта:

```
python -m db.import_json
```

Запуск через python -m выполняет файл как модуль пакета db, поэтому импорт from db.database import get_connection работает корректно независимо от текущего каталога запуска. При успешном выполнении команда завершается без сообщений об ошибке.

### *5.5.4. Проверка импортированных данных*

После импорта проверьте содержимое базы данных. В PowerShell запустите интерактивный Python и выполните:

```
import sqlite3
conn = sqlite3.connect("data/roomly.sqlite")
```

Проверьте количество записей в каждой таблице:

```
conn.execute("SELECT COUNT(*) FROM rooms").fetchone()
conn.execute("SELECT COUNT(*) FROM users").fetchone()
conn.execute("SELECT COUNT(*) FROM bookings").fetchone()
```

Ожидаемый результат – (3,), (1,) и (2,) соответственно: по числу записей в исходных JSON-файлах.

Проверьте содержимое таблицы rooms:

```
conn.execute("SELECT * FROM rooms").fetchall()
```

Ожидаемый результат:

```
[
(1, 'Аудитория 301', 30),
(2, 'Конференц-зал', 60),
(3, 'Компьютерный класс', 25)
]
```

Проверьте содержимое таблицы users:

```
conn.execute("SELECT * FROM users").fetchall()
```

Ожидаемый результат:

```
[
(1, 'User', 'user@example.com')
]
```

Проверьте содержимое таблицы bookings:

```
conn.execute("SELECT * FROM bookings").fetchall()
```

Ожидаемый результат:

```
[
(1, 1, 1, '2026-09-20', 0),
(2, 2, 1, '2026-09-21', 0)
]
```

Для проверки установленных связей выполните запрос, объединяющий данные трёх таблиц:

```
conn.execute("""
SELECT
bookings.id,
bookings.booking_date,
rooms.name,
users.name
FROM bookings
JOIN rooms
ON bookings.room_id = rooms.id
JOIN users
ON bookings.user_id = users.id;
""").fetchall()
```

Ожидаемый результат:

```
[
(1, '2026-09-20', 'Аудитория 301', 'User'),
(2, '2026-09-21', 'Конференц-зал', 'User')
]
```

Запрос JOIN объединяет данные нескольких связанных таблиц: для каждого бронирования выводятся дата, название помещения и имя пользователя. Подробно соединение таблиц рассматривается в подразделе 5.6.5.

После проверки закройте соединение:

```
conn.close()
```

## 5.6. Разработка SQL-запросов проекта

На данном этапе необходимо разработать SQL-запросы, которые реализуют функциональность информационной системы: выводить список объектов, находить подходящие варианты, собирать связанную информацию из нескольких таблиц, вычислять показатели. Именно такие запросы затем используются Django-представлениями для формирования веб-страниц.

**Примечание.** Запросы, приведённые в подразделах 5.6.2–5.6.9, выполняются для проекта Roomly и приводятся только в качестве образца, а не как руководство к деятельности: для индивидуального проекта состав и формулировки запросов определяются его предметной областью. Минимальный объём сохраняется – не менее двух запросов каждого вида и не менее 10 запросов в сумме; результаты запросов переносятся в веб-приложение (подраздел 5.7).

В результате выполнения данного этапа необходимо получить набор SQL-запросов, которые:

- работают с фактическими таблицами базы данных проекта;
- используют реальные данные проекта;
- решают задачи предметной области;
- могут быть использованы Django-представлениями;
- возвращают данные, которые можно отобразить в шаблоне сайта.

Порядок работы над каждым запросом:

```
SQL-запрос
↓
проверка в SQLite
↓
сохранение в sql/queries.sql
↓
получение результата в Django
↓
передача результата в шаблон
↓
отображение на сайте
```

Сначала каждый запрос проверяется непосредственно в базе данных, затем сохраняется в файл запросов проекта и только после этого переносится в Django-представление. Такой порядок позволяет убедиться в корректности запроса до его использования в коде приложения.

Ниже приведены запросы для проекта Roomly – их необходимо адаптировать к предметной области собственного проекта.

### *5.6.1. Подготовка файла SQL-запросов*

Перейдите в файл sql/queries.sql – в нём будут храниться проверенные запросы проекта. Для каждого запроса используйте комментарий с указанием его назначения, например:

```
-- Получение списка всех помещений
SELECT
id,
name,
capacity
FROM rooms
ORDER BY name;
```

Комментарий начинается с двух дефисов, не выполняется как SQL-команда и позволяет определить назначение запроса при просмотре файла.

Для проверки запросов используется SQLite CLI – интерактивная консоль SQLite. В PowerShell, из корневого каталога проекта, выполните:

```
sqlite3 data/roomly.sqlite
```

После появления приглашения sqlite> можно выполнять SQL-запросы.

### *5.6.2. Запрос 1. Список помещений*

Первая задача – получить список помещений, который будет использоваться на странице списка rooms-раздела сайта. Составьте запрос, выбирающий идентификатор, название и вместимость всех помещений с сортировкой по названию:

```
SELECT
id,
name,
capacity
FROM rooms
ORDER BY name;
```

В запросе:

- SELECT определяет поля, которые необходимо получить;
- FROM rooms указывает таблицу-источник;
- ORDER BY name сортирует помещения по названию.

Выполните запрос в SQLite CLI. Для тестовых данных проекта результат будет содержать:

```
1 | Аудитория 301 | 30
2 | Компьютерный класс | 25
3 | Конференц-зал | 60
```

После проверки добавьте запрос с комментарием в sql/queries.sql.

Данный запрос является основой страницы списка помещений: в подразделе 5.7.3 его результат будет передаваться в шаблон rooms/room_list.html.

### *5.6.3. Запрос 2. Поиск помещений по вместимости*

Следующая задача – найти помещения, подходящие для мероприятия с заданным количеством участников. Например, необходимо найти помещения вместимостью не менее 30 человек:

```
SELECT
id,
name,
capacity
FROM rooms
WHERE capacity >= 30
ORDER BY capacity;
```

Условие WHERE оставляет в результате только записи, удовлетворяющие указанному условию. Выполните запрос в SQLite CLI и проверьте результат:

```
1 | Аудитория 301 | 30
2 | Конференц-зал | 60
```

Сохраните запрос в sql/queries.sql с комментарием. Значение 30 выбрано здесь только для проверки: в веб-приложении оно будет заменено значением, полученным от пользователя (подраздел 5.7.4).

### *5.6.4. Запрос 3. Активные бронирования*

В предметной области Roomly отменённое бронирование не учитывается как активное. В базе данных это состояние представлено значением is_cancelled = 0. Составьте запрос, выводящий только действующие бронирования:

```
SELECT
id,
room_id,
user_id,
booking_date
FROM bookings
WHERE is_cancelled = 0
ORDER BY booking_date;
```

Выполните запрос в SQLite CLI, проверьте результат и сохраните запрос с комментарием в sql/queries.sql.

Результат такого запроса используется на странице списка бронирований (подраздел 5.7.3).

### *5.6.5. Запрос 4. Полная информация о бронировании*

Для страницы бронирований идентификаторов room_id и user_id недостаточно: пользователю нужны названия помещений, имена участников и другие понятные сведения. Сведения о бронировании, помещении и пользователе хранятся в разных таблицах, поэтому их необходимо объединить. Для объединения данных нескольких таблиц используется JOIN:

```
SELECT
bookings.id,
bookings.booking_date,
rooms.name AS room_name,
rooms.capacity,
users.name AS user_name,
users.email
FROM bookings
JOIN rooms
ON bookings.room_id = rooms.id
JOIN users
ON bookings.user_id = users.id
WHERE bookings.is_cancelled = 0
ORDER BY bookings.booking_date;
```

Уточнение JOIN ... ON указывает, по каким полям соединяются таблицы. Конструкция:

```
JOIN rooms ON bookings.room_id = rooms.id
```

соединяет каждое бронирование с помещением, а конструкция:

```
JOIN users ON bookings.user_id = users.id
```

– с пользователем. Ключевое слово AS задаёт псевдоним столбца результата: rooms.name AS room_name позволяет обращаться к полю по понятному имени в Python-коде и в шаблоне.

Выполните запрос в SQLite CLI. Результат:

```
1 | 2026-09-20 | Аудитория 301 | 30 | User | user@example.com
2 | 2026-09-21 | Конференц-зал | 60 | User | user@example.com
```

Сохраните запрос в sql/queries.sql. Это один из основных запросов проекта: он демонстрирует практическую пользу реляционной структуры – вместо

идентификаторов запрос возвращает готовую для отображения информацию из трёх связанных таблиц.

### *5.6.6. Запрос 5. Свободные помещения на выбранную дату*

Для системы бронирования важнейшая задача – определить помещения, которые можно забронировать на выбранную дату. Помещение свободно, если на эту дату у него нет активного бронирования. Такие задачи решаются с помощью подзапроса. Например, проверяется дата 2026-09-20:

```
SELECT
rooms.id,
rooms.name,
rooms.capacity
FROM rooms
WHERE rooms.id NOT IN (
SELECT bookings.room_id
FROM bookings
WHERE bookings.booking_date = '2026-09-20'
AND bookings.is_cancelled = 0
)
ORDER BY rooms.name;
```

Запрос состоит из двух частей. Внутренний запрос (в скобках) получает идентификаторы помещений, которые уже заняты на выбранную дату:

```
SELECT bookings.room_id
FROM bookings
WHERE bookings.booking_date = '2026-09-20'
AND bookings.is_cancelled = 0
```

Внешний запрос выбирает помещения, идентификаторы которых не входят в этот список – операция `NOT IN` исключает занятые помещения из результата. Выполните запрос в `SQLite CLI`: для тестовых данных на 2026-09-20 занята «Аудитория 301», поэтому в результат должны войти только:

```
Компьютерный класс
Конференц-зал
```

Сохраните запрос с комментарием в `sql/queries.sql`. Дата `'2026-09-20'` использована только для проверки: в веб-приложении она будет передаваться в запрос параметром из `Django` (подраздел 5.7.4).

### *5.6.7. Запрос 6. Количество бронирований*

Для информационного блока сайта полезно знать общее число действующих бронирований. Подсчёт выполняет агрегатная функция `COUNT()`:

```
SELECT
COUNT(*) AS bookings_count
```

```
FROM bookings
WHERE is_cancelled = 0;
```

Агрегатная функция обрабатывает набор строк и возвращает одно значение: COUNT(*) подсчитывает количество записей, удовлетворяющих условию WHERE. Выполните запрос в SQLite CLI: для тестовых данных результат – 2. Сохраните запрос в sql/queries.sql.

Такой результат может отображаться, например, в информационном блоке сайта: «Активных бронирований: 2».

*5.6.8. Запрос 7. Количество бронирований по помещениям*

Для страницы статистики необходимо определить количество бронирований каждого помещения. Задача объединяет соединение таблиц, группировку и подсчёт:

```
SELECT
rooms.name AS room_name,
COUNT(bookings.id) AS bookings_count
FROM rooms
LEFT JOIN bookings
ON rooms.id = bookings.room_id
AND bookings.is_cancelled = 0
GROUP BY rooms.id, rooms.name
ORDER BY bookings_count DESC;
```

В запросе:

- LEFT JOIN подключает бронирования к помещениям; в отличие от обычного JOIN, помещения без бронирований не исключаются из результата;
- условие is_cancelled = 0 в блоке ON учитывает только активные бронирования;
- GROUP BY rooms.id, rooms.name формирует группы по одному на каждое помещение;
- COUNT(bookings.id) подсчитывает количество бронирований в каждой группе.

Выполните запрос в SQLite CLI. Для тестовых данных результат:

```
Аудитория 301 | 1
Конференц-зал | 1
Компьютерный класс | 0
```

Помещение без активных бронирований получило значение 0, а не исчезло из списка, – именно для этого использован LEFT JOIN. Сохраните запрос в sql/queries.sql.

*5.6.9. Запрос 8. Помещения с количеством бронирований больше заданного*

Ещё одна задача статистики – найти помещения, пользующиеся наибольшим спросом. Например, вывести помещения, забронированные более одного раза. Фильтрацию сформированных групп выполняет HAVING:

```
SELECT
rooms.name AS room_name,
COUNT(bookings.id) AS bookings_count
FROM rooms
JOIN bookings
ON rooms.id = bookings.room_id
WHERE bookings.is_cancelled = 0
GROUP BY rooms.id, rooms.name
HAVING COUNT(bookings.id) > 1
ORDER BY bookings_count DESC;
```

Различие между `WHERE` и `HAVING` принципиально: `WHERE` фильтрует отдельные записи до группировки, `HAVING` – группы после группировки. Порядок обработки частей запроса:

```
FROM / JOIN
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
ORDER BY
```

Выполните запрос в `SQLite CLI`. Для текущего набора данных результат будет пустым: каждое помещение имеет не более одного активного бронирования. Пустой результат является корректным – условию `HAVING` не удовлетворяет ни одна группа.

**Примечание.** Запрос демонстрирует работу `HAVING` на данных проекта. Его практический смысл проявится по мере накопления данных: добавьте в таблицу `bookings` несколько дополнительных записей и проверьте, как изменится результат.

Для добавления данных в SQLite:

1. Откройте терминал VS Code в корневой папке проекта Roomly.
2. Подключитесь к базе данных:

```
sqlite3 data/roomly.sqlite
```

3. Добавьте бронирования для проверки запроса с HAVING:

```
INSERT INTO bookings
    (id, room_id, user_id, booking_date, is_cancelled)
VALUES
    (3, 1, 1, '2026-10-10', 0),
    (4, 1, 1, '2026-10-11', 0),
    (5, 2, 1, '2026-10-12', 0);
```

Здесь используются существующие помещения и пользователь. Идентификаторы новых бронирований должны быть свободны.

4. Проверьте, что записи добавлены:

```
SELECT * FROM bookings;
```

6. Выйдите из SQLite:

```
.exit
```

## 5.7. Получение результатов SQL-запросов в Django

*5.7.1. Вводная часть*

После разработки и проверки запросов необходимо включить их в веб-приложение. SQL-запрос становится внутренним механизмом страницы: представление выполняет запрос к базе данных, получает результат и передаёт его в шаблон. Путь данных в Django-приложении:

```
База данных SQLite
↓
SQL-запрос
↓
Django view
↓
context
↓
Django template
↓
веб-страница
```

Реализуйте несколько пользовательских сценариев – не менее 5:

1. Просмотр списка помещений;
2. Просмотр активных бронирований;
3. Поиск свободных помещений на выбранную дату;
4. Просмотр статистики по помещениям.

На страницах сайта пользователь работает с разделами предметной области, а не с SQL-командами: названия пунктов меню и страниц формулируются в терминах предметной области («Помещения», «Бронирования», «Статистика»), а SQL-запросы остаются внутренним механизмом получения данных.

*5.7. 1. Настройка получения именованных результатов SQLite*

По умолчанию библиотека sqlite3 возвращает строки результата в виде кортежей, и обращение к значениям возможно только по индексам:

```
[
(1, "Аудитория 301", 30),
(2, "Конференц-зал", 60),
]
```

В шаблоне пришлось бы использовать индексы: {{ room.0 }}, {{ room.1 }} и так далее. Такой способ делает шаблон зависимым от порядка полей в SQL-запросе: при изменении списка столбцов запроса шаблон придётся изменять вместе с ним. Чтобы обращаться к полям результата по именам, настройте фабрику строк при создании соединения:

```
conn = sqlite3.connect(DB_PATH)
conn.row_factory = sqlite3.Row
```

После настройки `conn.row_factory = sqlite3.Row` каждая строка результата становится объектом, допускающим доступ к полям по имени: `room["name"]` в Python-коде и `{{ room.name }}` в шаблоне. Именно этот вариант используется во всех представлениях далее.

Откройте файл `rooms/views.py` и добавьте в начало файла импорт модуля `sqlite3` и путь к базе данных:

```
import sqlite3
from django.shortcuts import render

DB_PATH = "data/roomly.sqlite"
```

### *5.7.2. Отображение списка помещений*

Реализуйте представление, которое получает список помещений из SQLite и передаёт его в существующий шаблон. Откройте `rooms/views.py` и реализуйте представление `room_list`:

```
def rooms(request):
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    rooms = conn.execute("""
        SELECT
        id,
        name,
        capacity
        FROM rooms
        ORDER BY name;
        """).fetchall()
    conn.close()
    return render(
        request,
        "rooms/room_list.html",
        {"rooms": rooms},
    )
```

В представлении:

- `sqlite3.connect(DB_PATH)` устанавливает соединение с базой данных;
- `row_factory = sqlite3.Row` обеспечивает доступ к полям результата по именам;
- `execute()` выполняет SQL-запрос, `fetchall()` получает все строки результата;
- полученный список передаётся в шаблон через `context`.

Убедитесь, что в rooms/urls.py определён маршрут списка помещений:

```
path("", views.room_list, name="list"),
```

Откройте шаблон rooms/templates/rooms/room_list.html и выведите полученные данные в цикле:

```
{% for room in rooms %}
<article class="room-card">
<h2>{{ room.name }}</h2>
<p>Вместимость: {{ room.capacity }} человек</p>
</article>
{% empty %}
<p>Помещения не найдены.</p>
{% endfor %}
```

Тег {% empty %} выводит содержимое, если список пуст, – шаблон корректно обрабатывает отсутствие данных.

Запустите сервер разработки из корня проекта и откройте страницу списка помещений:

```
python manage.py runserver
```

На странице должны отображаться данные из таблицы rooms:

```
Аудитория 301
Вместимость: 30 человек
Компьютерный класс
Вместимость: 25 человек
Конференц-зал
Вместимость: 60 человек
```

Убедитесь, что данные на странице больше не заданы статически в HTML, а поступают из базы данных: измените запись в таблице rooms (например, название помещения) и проверьте, что после обновления страница отображает актуальные данные.

**Примечание**: Для изменения название помещения выполните:
1. Откройте терминал VS Code в корневой папке проекта Roomly.
2. Подключитесь к базе данных:

```
sqlite3 data/roomly.sqlite
```

3. Внесите изменения в записи БД

```
UPDATE rooms
SET name = 'Аудитория 301 — обновлено'
WHERE id = 1;
```

4. Проверьте результат:

```
SELECT * FROM rooms WHERE id = 1;
```

6. Выйдите из SQLite:

```
.exit
```

7. Обновите страницу Roomly в браузере. Название помещения должно измениться, если страница получает данные из базы данных через SQL-запрос, а не содержит статический HTML.

**Не запускайте повторно** приведённый `INSERT` без проверки: при повторном добавлении возникнет ошибка, если идентификаторы уже заняты.

### *5.7.3. Отображение активных бронирований*

Реализуйте страницу активных бронирований аналогичным образом. Используется запрос из подраздела 5.6.5: соединение таблиц позволяет вывести понятные пользователю данные вместо идентификаторов. Откройте `bookings/views.py` и реализуйте представление:

```
import sqlite3
from django.shortcuts import render

DB_PATH = "data/roomly.sqlite"

def bookings(request):
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    bookings_data = conn.execute("""
        SELECT
        bookings.id
,
        bookings.booking_date,
        rooms.name
 AS room_name,
        rooms.capacity,
        users.name
 AS user_name
        FROM bookings
        JOIN rooms
        ON bookings.room_id = rooms.id
        JOIN users
        ON bookings.user_id = users.id
        WHERE bookings.is_cancelled = 0
        ORDER BY bookings.booking_date;
        """).fetchall()
    conn.close()
    return render(
        request,
        "bookings/booking_list.html",
        {"bookings": bookings_data},
    )
```

Убедитесь, что в `bookings/urls.py` определён маршрут:

```
path("", views.bookings, name="bookings"),
```

Откройте шаблон `bookings/templates/bookings/booking_list.html` и выведите данные:

```
{% for booking in bookings %}
<article class="booking-card">
<h2>{{ booking.room_name }}</h2>
<p>Дата: {{ booking.booking_date }}</p>
<p>Вместимость: {{ booking.capacity }} человек</p>
<p>Пользователь: {{ booking.user_name }}</p>
</article>
{% empty %}
<p>Активных бронирований нет.</p>
{% endfor %}
```

Запустите сервер разработки и откройте страницу бронирований. На странице должны отображаться данные, собранные из трёх таблиц:

```
Аудитория 301
Дата: 2026-09-20
Вместимость: 30 человек
Пользователь: User
```

Связи, использованные запросом:

```
bookings
│
├── room_id → rooms.id
│
└── user_id → users.id
```

### *5.7.4. Создание страницы поиска свободных помещений*

Реализуйте самый содержательный сценарий `Roomly` – поиск помещения, свободного на выбранную дату. Пользователь указывает дату в форме, приложение возвращает помещения, у которых нет активных бронирований на эту дату.

Проверьте запрос из подраздела 5.6.6 в `SQLite CLI` и сохраните его с комментарием в `sql/queries.sql`.

Добавьте в `rooms/urls.py` маршрут поиска:

```
path("available/", views.available_rooms, name="available"),
```

В `rooms/views.py` реализуйте представление:

```
def available_rooms(request):
    booking_date = request.GET.get("date")
    rooms = []
    if booking_date:
        conn = sqlite3.connect(DB_PATH)
        conn.row_factory = sqlite3.Row
```

```
        rooms = conn.execute("""
            SELECT
            rooms.id
,
            rooms.name
,
            rooms.capacity
            FROM rooms
            WHERE rooms.id
 NOT IN (
            SELECT bookings.room_id
            FROM bookings
            WHERE bookings.booking_date = ?
            AND bookings.is_cancelled = 0
            )
            ORDER BY rooms.name
;
            """, [booking_date]).fetchall()
        conn.close()
    return render(
        request,
        "rooms/available.html",
        {
        "rooms": rooms,
        "booking_date": booking_date,
        },
    )
```

В представлении:

- `request.GET.get("date")` получает дату из параметра URL, например `/rooms/available/?date=2026-09-20`;
- если дата не указана, запрос к базе не выполняется – страница отображает только форму поиска;
- в тексте SQL-запроса используется заполнитель ?, а значение даты передаётся отдельно, списком `[booking_date]`.

Такой способ передачи значений называется параметризованным запросом. Значение никогда не встраивается в текст SQL-запроса, поэтому данные пользователя не могут исказить запрос.

Создайте шаблон `rooms/templates/rooms/available.html`:

```
<h1>Свободные помещения</h1>

<form method="get">
<label for="date">Дата:</label>
```

```
<input
type="date"
id="date"
name="date"
value="{{ booking_date }}"
>
<button type="submit">Найти</button>
</form>

{% if booking_date %}
<h2>Дата: {{ booking_date }}</h2>
{% for room in rooms %}
<article class="room-card">
<h3>{{ room.name }}</h3>
<p>Вместимость: {{ room.capacity }} человек</p>
</article>
{% empty %}
<p>На выбранную дату свободных помещений нет.</p>
{% endfor %}
{% endif %}
```

Форма отправляет выбранную дату методом `GET`; атрибут `value="{{ booking_date }}"` сохраняет выбранный пользователем вариант после отправки.

**Примечание.** `HTML`-код, приведённый в подразделе, содержит только содержимое блока `content`. Создаваемый шаблон должен расширять имеющийся базовый шаблон `base.html`: подключите базовый шаблон инструкцией `{% extends %}` (как в шаблонах, созданных в ПР5–ПР6) и поместите приведённую разметку внутрь блока `content` – шапка, навигация и стили сайта подключатся из `base.html` автоматически.

Откройте страницу `/rooms/available/`, выберите дату и отправьте форму. После отправки должна открыться страница с помещениями, которые не имеют активных бронирований на выбранную дату.

### *5.7.5. Создание страницы статистики*

Реализуйте страницу статистики на основе запроса из подраздела 5.6.8: количество активных бронирований каждого помещения. Статистика относится к предметной области бронирований, поэтому маршрут и представление размещаются в приложении `bookings`.

Добавьте в `bookings/urls.py` маршрут:

```
path("statistics/", views.statistics, name="statistics"),
```

В `bookings/views.py` реализуйте представление:

```
def statistics(request):
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
```

```
    statistics_data = conn.execute("""
        SELECT
        rooms.name
 AS room_name,
        COUNT(bookings.id
) AS bookings_count
        FROM rooms
        LEFT JOIN bookings
        ON rooms.id
 = bookings.room_id
        AND bookings.is_cancelled = 0
        GROUP BY rooms.id
, rooms.name
        ORDER BY bookings_count DESC;
        """).fetchall()
    conn.close()
    return render(
        request,
        "bookings/statistics.html",
        {"statistics": statistics_data},
    )
```

Создайте шаблон `bookings/templates/bookings/statistics.html`:

```
<h1>Статистика бронирований</h1>
{% for item in statistics %}
<article class="statistics-item">
<h2>{{ item.room_name }}</h2>
<p>Количество бронирований: {{ item.bookings_count }}</p>
</article>
{% empty %}
<p>Данные отсутствуют.</p>
{% endfor %}
```

**Примечание.** Приведённый `HTML`-код содержит только содержимое блока `content`. Создаваемый шаблон должен расширять имеющийся базовый шаблон `base.html`: подключите базовый шаблон инструкцией `{% extends %}` и поместите разметку внутрь блока `content`.

Откройте страницу `/bookings/statistics/` – на ней должно отображаться количество активных бронирований для каждого помещения, включая помещения без бронирований.

### *5.7. 6. Организация навигации*

После добавления новых функций включите их в существующую навигацию сайта. Функции размещаются в разделах предметной области, а не представляются пользователю как отдельные `SQL`-операции:

- «Помещения» – просмотр всех помещений и поиск свободных помещений на выбранную дату;
- «Бронирования» – просмотр активных бронирований;
- «Статистика» – просмотр статистики бронирований.

Общая навигация сайта должна обеспечивать переходы между основными разделами:

```
Главная | Помещения | Бронирования | Статистика
```

Откройте файл навигации `homepage/templates/homepage/includes/navigation.html` и проверьте, что ссылки используют именованные URL:

```
<nav>
<a href="{% url 'homepage:index' %}">Главная</a>
<a href="{% url 'rooms:list' %}">Помещения</a>
<a href="{% url 'bookings:bookings' %}">Бронирования</a>
<a href="{% url 'bookings:statistics' %}">Статистика</a>
</nav>
```

Маршруты, на которые опирается меню, реализованы в предыдущих подразделах:

- `rooms:list` – список помещений (подраздел 5.7.2);
- `bookings:bookings` – активные бронирования (подраздел 5.7.3);
- `rooms:available` – поиск свободных помещений (подраздел 5.7.4);
- `bookings:statistics` – статистика бронирований (подраздел 5.7.5).

Пространства имён задаются переменной `app_name` в `urls.py` каждого приложения. Если в вашем проекте используются другие имена маршрутов, в меню необходимо указать фактические имена, определённые в `urls.py`.

Убедитесь, что навигационный блок подключается в общем шаблоне `homepage/templates/homepage/base.html`:

```
{% include "homepage/includes/navigation.html" %}
```

Благодаря этому навигация отображается на всех страницах, использующих `base.html`.

Поиск свободных помещений не обязательно включать в общее меню – это функция раздела «Помещения». Добавьте на страницу списка помещений ссылку:

```
<a href="{% url 'rooms:available' %}">
Найти свободное помещение
</a>
```

Структура раздела «Помещения» получается следующей:

```
Помещения
├── Все помещения
└── Свободные помещения
```

Запустите сервер разработки и последовательно проверьте:

- переход на страницу «Помещения» и отображение списка помещений;
- переход по ссылке «Найти свободное помещение»;
- выбор даты и результат поиска свободных помещений;
- переход в раздел «Бронирования» и отображение активных бронирований;
- переход в раздел «Статистика» и отображение количества бронирований по помещениям.

Все переходы должны выполняться через именованные URL Django. Прямое указание адресов страниц в шаблонах, например href="/rooms/", не используется.

SQL-запросы при этом не отображаются в пользовательском интерфейсе: пользователь работает с разделами «Помещения», «Бронирования» и «Статистика», а Django выполняет соответствующие SQL-запросы к базе данных.

## 5.8. Завершение работы

### *5.8.1 Проверка работы SQL-функциональности*

Перед завершением работы проверьте весь путь данных – от таблиц базы данных до веб-страниц.

**Примечание.** Приведённые ниже проверки и команды являются учебным примером для проекта Roomly: в реальном проекте их необходимо адаптировать к собственному учебному проекту – использовать имя своей базы данных, свои таблицы, свои запросы и адреса своих страниц.

**Проверка 1. База данных**

Откройте data/roomly.sqlite в SQLite Viewer и убедитесь, что существуют таблицы rooms, users, bookings с импортированными данными.

**Проверка 2. Файл запросов**

Откройте sql/queries.sql и проверьте наличие разработанных запросов. Минимальный набор должен включать:

- получение списка помещений;
- фильтрацию (WHERE);
- соединение таблиц (JOIN);
- агрегатный запрос (COUNT);
- группировку (GROUP BY).

**Проверка 3. SQLite CLI**

Выполните каждый запрос относительно базы `data/roomly.sqlite` и убедитесь, что результат соответствует назначению запроса.

**Проверка 4. `Django`**

Запустите сервер разработки и проверьте страницы:

```
/rooms/
/bookings/
/rooms/available/
/bookings/statistics/
```

**Проверка 5. Веб-интерфейс**

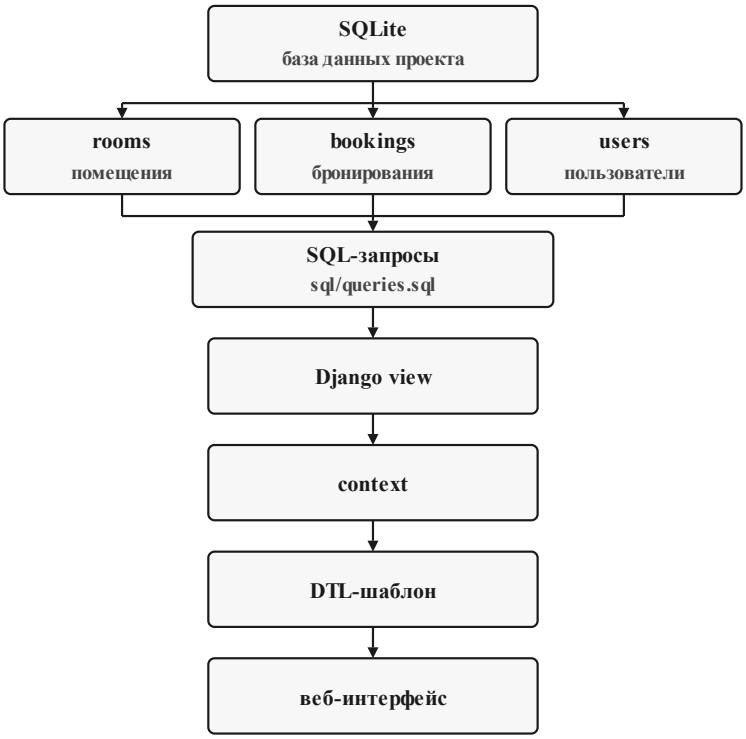
Убедитесь, что данные на страницах соответствуют данным `SQLite`. Например, если в таблице `rooms` есть запись `(1, 'Аудитория 301', 30)`, на странице помещений должно отображаться: «Аудитория 301, вместимость: 30 человек».

*5.8.2. Требования к результату*

После выполнения подраздела 5.6 в проекте должен работать фрагмент веб-приложения, получающий данные из `SQLite`. Необходимо обеспечить:

1. Наличие файла `sql/queries.sql` с проверенными запросами;
2. Использование `select` и `where`;
3. Использование `join` для получения связанных данных;
4. Использование агрегатной функции, например `count`();
5. Использование `group by`;
6. Получение результатов `sql`-запросов в `django`-представлениях;
7. Использование `sqlite3.row` для обращения к полям результата по именам;
8. Передачу результатов запросов в шаблоны через `context`;
9. Отображение результатов запросов на веб-страницах;
10. Реализацию поиска свободных помещений по выбранной дате;
11. Реализацию страницы статистики;
12. Наличие навигации между реализованными функциями.

Итоговая схема работы данных:




Минимальный объём SQL-части: не менее 10 запросов, в том числе не менее 2 запросов каждого вида – выборка списков, фильтрация (WHERE), соединение таблиц (JOIN), агрегатные функции (COUNT), группировка (GROUP BY). Запросы, приведённые в методичке, являются примерами и служат только образцом, а не руководством к деятельности: состав и количество запросов определяются предметной областью проекта. Количество запросов сверх минимального определяется функциональностью проекта. Приоритетом является наличие работающих пользовательских сценариев, в которых данные из SQLite действительно используются веб-приложением: результаты запросов получают Django-представления и отображаются на страницах сайта (подраздел 5.7).

### 5.8.3 Обновление README.md

В корне проекта должен находиться файл README.md, описывающий текущее состояние проекта. После выполнения ПР7 дополните описание разделом о слое базы данных:

- спроектированную структуру базы данных: перечень таблиц с полями и типами данных;
- первичные и внешние ключи;
- файл sql/schema.sql с командами создания таблиц;
- скрипт db/import_json.py переноса данных из JSON;
- перечень SQL-запросов в файле sql/queries.sql;
- получение данных из SQLite в Django-представлениях;
- страницы, на которых данные из базы отображаются пользователю.

Пример раздела о текущем этапе для проекта Roomly:

```
## Текущий этап разработки

На ПР7 данные Roomly перенесены из JSON в базу данных SQLite:

- создана база данных data/roomly.sqlite;
- таблицы созданы с помощью sql/schema.sql;
- данные перенесены из JSON скриптом db/import_json.py;
- SQL-запросы сохранены в sql/queries.sql;
- Django-представления получают данные из SQLite;
- страницы списка помещений, бронирований, поиска
свободных помещений и статистики работают с базой данных.

На следующих практических работах база данных будет
подключена к Django с использованием Django ORM.
```

**Примечание.** Для индивидуального проекта README должен быть адаптирован под выбранную предметную область: замените описание Roomly, перечень сущностей, таблиц, полей и примеры SQL-запросов на данные собственного проекта.

### 5.8.4 Проверка и фиксация результата

#### 5.8.4.1. Проверка ранее разработанной части проекта

ПР7 не должна нарушать работоспособность проекта, созданного на предыдущих практических работах. Выполните проверки:

```
python manage.py check
pytest -v
flake8 . --extend-exclude=venv
```

При необходимости запустите Django-сервер разработки и убедитесь, что страницы, созданные в ПР5–ПР6, продолжают открываться.

#### 5.8.4.2. Фиксация результата в Git

После завершения всех изменений зафиксируйте результат. Проверьте состояние репозитория и состав изменений:

```
git status
git diff
```

Добавьте файлы в индекс и создайте коммит:

```
git add .
git commit -m "PR7: add SQLite database and SQL queries"
```

При использовании удалённого репозитория выполните:

```
git push
```

Коммит ПР7 должен начинаться с префикса PR7:.

## 6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

В результате выполнения ПР7 индивидуальный проект получает слой реляционного хранения данных: базу SQLite, набор SQL-запросов и получение данных в Django-представлениях. Используйте этот перечень для самопроверки перед переходом к контролю: каждый пункт должен быть подтверждён демонстрацией или содержимым репозитория.

После выполнения ПР7 в проекте должны быть:

- спроектирована структура базы данных предметной области;
- создан файл SQLite и таблицы средствами SQL (`sql/schema.sql`);
- определены первичные и внешние ключи;
- созданы модуль подключения db/database.py и скрипт импорта `db/import_json.py`;
- данные перенесены из JSON-файлов в таблицы базы;
- создан файл `sql/queries.sql` с проверенными запросами;
- разработано не менее 10 `SQL`-запросов, в том числе не менее двух запросов каждого вида (выборка списков, фильтрация `WHERE`, соединение таблиц `JOIN`, агрегатные функции, группировка `GROUP BY`), результаты которых перенесены в веб-приложение;
- реализованы запросы `SELECT` и `WHERE` для списков и поиска;
- реализованы запросы с `JOIN` для связанных данных;
- реализованы агрегатные запросы и группировка (`GROUP BY`);
- результаты SQL-запросов получаются в Django-представлениях;
- данные отображаются на веб-страницах проекта;
- реализован поиск свободных помещений по выбранной дате;
- реализована страница статистики бронирований;
- настроена навигация между разделами сайта;
- обновлён README.md;
- выполнены проверки проекта;
- изменения сохранены в Git.

Архитектура данных проекта после выполнения работы показана на рис. 4. Веб-часть Roomly, созданная в ПР5–ПР6, сохраняется: представления получают данные из базы SQLite средствами sqlite3, а JSON-файлы остаются в проекте как исходные данные, перенесённые в базу.

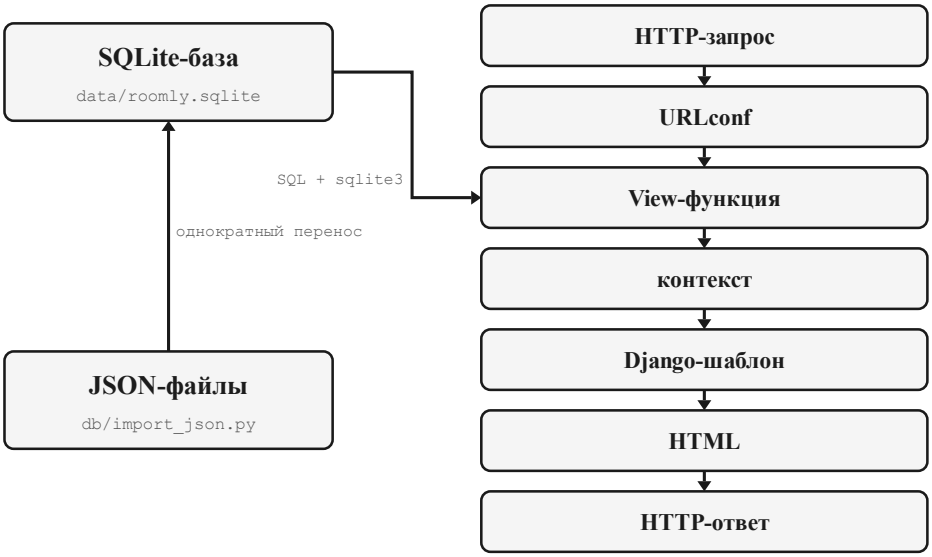



Рисунок 4 – Архитектура проекта Roomly после ПР7

**Примечание.** На данном этапе база данных существует отдельно от Django ORM. Подключение базы данных к Django-моделям выполняется на следующих практических работах.

## 7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ

Защита практической работы проводится в форме демонстрации работоспособности базы данных и устного ответа на контрольные вопросы. При защите необходимо продемонстрировать:

- структуру созданной базы данных: таблицы и поля;
- первичные и внешние ключи;
- данные, перенесённые из JSON-файлов;
- файлы `sql/schema.sql` и `sql/queries.sql`;
- пример запроса с `WHERE`;
- пример запроса с `JOIN`;
- агрегатный запрос и запрос с `GROUP BY`;
- получение данных из SQLite в Django-представлении;
- страницы сайта, отображающие данные из базы;
- Python-код работы с SQLite;
- параметризованный SQL-запрос;
- обновлённый README.md;
- результаты выполнения проверок;
- Git-коммит с результатами ПР7.

### 7.1. Чек-лист выполнения материалов Яндекс Практикума

Студент предъявляет преподавателю результат выполнения следующих материалов раздела 05 «Базы данных и Django ORM». Отмечайте пункты по мере

выполнения – чек-лист используется и при самопроверке, и при контроле со стороны преподавателя.

| № | Материал Яндекс Практикума | Выполнено |
|---|---|---|
| 1 | Введение в спринт | ☐ |
| 2 | Базы данных: реляционные и не только | ☐ |
| 3 | SQL: язык структурированных запросов | ☐ |
| 4 | Первые запросы, первые результаты | ☐ |
| 5 | Работа с базой данных из Python | ☐ |
| 6 | Получение данных | ☐ |
| 7 | Практика запросов | ☐ |
| 8 | Сортировка, ограничение и сдвиг выборки | ☐ |
| 9 | Агрегирующие функции | ☐ |
| 10 | Группировка записей в выборке | ☐ |

При защите студент должен быть готов подтвердить выполнение материалов курса и продемонстрировать результат соответствующих заданий в проекте.

### 7.2. Чек-лист выполнения индивидуального проекта

Чек-лист доработки индивидуального проекта:

| № | Требование | Выполнено |
|---|---|---|
| 1 | Проект ПР1–ПР6 сохранён и продолжен | ☐ |
| 2 | Определены сущности предметной области | ☐ |
| 3 | Определены таблицы | ☐ |
| 4 | Определены поля и типы данных | ☐ |
| 5 | Определены первичные ключи | ☐ |
| 6 | Определены необходимые внешние ключи | ☐ |
| 7 | Создан файл базы данных SQLite | ☐ |
| 8 | Таблицы созданы средствами SQL (sql/schema.sql) | ☐ |
| 9 | Создан модуль подключения db/database.py | ☐ |
| 10 | Включена проверка внешних ключей (PRAGMA foreign_keys = ON) | ☐ |
| 11 | Создан скрипт переноса данных db/import_json.py | ☐ |
| 12 | Данные перенесены из JSON-файлов | ☐ |
| 13 | Импортированные данные проверены | ☐ |
| 14 | Создан файл sql/queries.sql | ☐ |
| 15 | Запросы проверены в SQLite CLI | ☐ |
| 16 | Использованы SELECT и WHERE | ☐ |
| 17 | Использован JOIN для связанных данных | ☐ |
| 18 | Использованы агрегатные функции | ☐ |
| 19 | Выполнена группировка (GROUP BY) | ☐ |
| 20 | Выполнена фильтрация групп (HAVING) | ☐ |
| 21 | Значения передаются параметрами ? | ☐ |
| 22 | Использован sqlite3.Row в представлениях | ☐ |
| 23 | Список помещений выводится из SQLite | ☐ |
| 24 | Список бронирований выводится из SQLite | ☐ |
| 25 | Реализован поиск свободных помещений по дате | ☐ |
| 26 | Реализована страница статистики | ☐ |

| № | Требование | Выполнено |
|---|---|---|
| 27 | Настроена навигация между разделами | ☐ |
| 28 | Обновлён README.md | ☐ |
| 29 | Выполнены проверки проекта | ☐ |
| 30 | Изменения сохранены в Git с меткой PR7: | ☐ |
| 31 | Разработано не менее 10 `SQL`-запросов, в том числе не менее двух каждого вида (`WHERE`, `JOIN`, агрегатные функции, `GROUP BY`) | ☐ |

### 7.3. Что необходимо показать при защите

На компьютере необходимо открыть проект и последовательно продемонстрировать:

1. Структуру базы данных. Показать файл `data/roomly.sqlite` в SQLite Viewer: таблицы rooms, users, bookings и их поля.
2. Первичные и внешние ключи. Показать файл sql/schema.sql и объяснить назначения ключей:

```
id INTEGER PRIMARY KEY
FOREIGN KEY (room_id) REFERENCES rooms(id)
```

3. Данные проекта в таблицах. Показать содержимое таблиц и объяснить, как данные попали в базу: перенос из JSON-файлов скриптом db/import_json.py.
4. Запрос с `WHERE`. Показать файл `sql/queries.sql` и выполнить запрос поиска помещений по вместимости:

```
SELECT id, name, capacity
FROM rooms
WHERE capacity >= 30
ORDER BY capacity;
```

5. Запрос с `JOIN`. Выполнить запрос получения полной информации о бронированиях:

```
SELECT bookings.booking_date, rooms.name AS room_name,
users.name AS user_name
FROM bookings
JOIN rooms ON bookings.room_id = rooms.id
JOIN users ON bookings.user_id = users.id
WHERE bookings.is_cancelled = 0;
```

6. Поиск свободных помещений. Выполнить запрос с подзапросом NOT IN для выбранной даты:

```
SELECT name FROM rooms
WHERE rooms.id NOT IN (
SELECT bookings.room_id FROM bookings
WHERE bookings.booking_date = '2026-09-20'
AND bookings.is_cancelled = 0);
```

7. Агрегатный запрос и `GROUP BY`. Выполнить запрос статистики бронирований:

```
SELECT rooms.name, COUNT(bookings.id) AS bookings_count
FROM rooms
LEFT JOIN bookings ON rooms.id = bookings.room_id
AND bookings.is_cancelled = 0
GROUP BY rooms.id, rooms.name;
```

8. Python-код работы с SQLite. Показать `db/database.py`, `db/import_json.py` и параметризованный запрос в представлении:

```
conn.execute(query, [booking_date])
```

9. Страницы проекта с данными из SQLite. Открыть `/rooms/`, `/bookings/`, `/rooms/available/`, `/bookings/statistics/` и показать, что данные отображаются из базы.

10. Обновлённый `README.md`. Показать разделы о базе данных, SQL-запросах и текущем состоянии проекта.

11. Результаты проверок. Выполнить команды и показать отсутствие ошибок:

```
python manage.py check
pytest -v
flake8 . --extend-exclude=venv
```

12. Git-коммит ПР7. Показать коммит с меткой:

```
PR7: add SQLite database and SQL queries
```

**Примечание.** Приведённые в подразделе команды, запросы и адреса страниц являются учебным примером для проекта `Roomly`: при защите демонстрируются аналогичные материалы, адаптированные к предметной области собственного учебного проекта.

### 7.4. Критерии оценки

Практическая работа считается выполненной при соблюдении следующих условий.

| № | Критерий | Содержание |
|---|---|---|
| 1 | Материалы Практикума | Выполнены предусмотренные материалы раздела 05 |
| 2 | База данных | Создан рабочий файл SQLite; структура соответствует предметной области |
| 3 | Таблицы и ключи | Таблицы созданы средствами SQL; определены первичные и внешние ключи |
| 4 | Перенос данных | Данные проекта перенесены из JSON-файлов в таблицы базы |
| 5 | SQL-запросы | Файл queries.sql содержит запросы, решающие задачи предметной области; запросы проверены; разработано не менее 10 запросов, в том числе не менее двух каждого вида |
| 6 | Связанные данные | Использованы JOIN, агрегатные функции и GROUP BY |
| 7 | Django-интеграция | Результаты запросов получаются в представлениях и отображаются на страницах проекта |
| 8 | Работа из Python | Созданы модуль подключения и скрипт импорта; используется sqlite3.Row |
| 9 | Параметризованные запросы | Значения передаются через параметры ?; f-строки для подстановки значений не используются |

| № | Критерий | Содержание |
|---|---|---|
| 10 | Работоспособность | Функциональность ПР1–ПР6 не нарушена; выполнены проверки python manage.py check, pytest, flake8 |
| 11 | README | Документация обновлена и соответствует текущему состоянию проекта |
| 12 | Git | Изменения зафиксированы коммитом с меткой PR7: |

При частичном выполнении практической работы недостающие элементы могут быть представлены в ходе контроля в виде отдельных SQL-запросов или фрагментов Python-кода с пояснением их назначения.

### 7.5. Требования к отчету

Объем отчета – 1–2 страницы, не считая титульной информации. Отдельное подробное описание каждого упражнения Яндекс Практикума не требуется. Отчет должен содержать:

1. Общие сведения:

- наименование образовательной организации;
- дисциплину;
- номер практической работы;
- тему практической работы;
- учебную группу;
- ФИО студента;
- вариант.

2. Цель работы.

Кратко указать цель освоения основ баз данных и SQL и их применения в индивидуальном проекте.

3. Выполненные материалы Яндекс Практикума:

Привести перечень выполненных материалов ПР7 с отметкой о выполнении:

| Раздел | Выполненные темы |
|---|---|
| Раздел 05. «Базы данных и Django ORM» | Базы данных: реляционные и не только; SQL: язык структурированных запросов; первые запросы; работа с базой данных из Python; получение данных; практика запросов; сортировка, ограничение и сдвиг выборки; агрегирующие функции; группировка записей в выборке |

4. Доработка индивидуального проекта. Указать:

- спроектированную структуру базы данных;
- созданные таблицы и ключи;
- перенос данных из JSON-файлов;
- файлы `sql/schema.sql` и `sql/queries.sql`;
- модули `db/database.py` и `db/import_json.py`;
- SQL-запросы предметной области;
- страницы проекта, отображающие данные из SQLite;
- обновлённый `README.md`.

5. Результат работы.

Указать созданные объекты базы данных, перечень SQL-запросов и реализованные страницы проекта.

6. Проверка. Привести результаты:

```
python manage.py check;
pytest -v;
flake8 . --extend-exclude=venv.
```

7. Git. Указать созданный коммит ПР7.

8. Вывод.Сделать краткий вывод о результатах выполнения практической работы.

К работе прилагаются файл базы данных `data/roomly.sqlite`, файлы `sql/schema.sql` и `sql/queries.sql`, модули `db/database.py` и `db/import_json.py`, обновлённый `README.md` и Git-репозиторий проекта.

Минимальный состав материалов для сдачи:

```
room_booking/
├── data/
│   └── roomly.sqlite
├── sql/
│   ├── schema.sql
│   └── queries.sql
├── db/
│   ├── __init__.py
│   ├── database.py
│   └── import_json.py
├── ...
└── README.md
```

### 7.6. Таблица оценивания индивидуального проекта

За выполнение работы над индивидуальным проектом (без учёта прохождения курса Яндекс Практикума) студент получает **1,5 балла**. Для оценивания используется приведённая ниже таблица из 25 точечных показателей.

Оценивание проводится в форме демонстрации: студент показывает указанные элементы проекта и отвечает на вопросы преподавателя. Опрос проводится выборочно по пунктам таблицы. Демонстрация длится **не более 5 минут** и **завершается на первом неправильном ответе** либо его отсутствии; по результатам опроса за студентом закрепляется набранное на этот момент число баллов.

**Примечание**. В таблице приведены примеры умений и практических задач на материале учебной предметной области Roomly. Демонстрация сформированных умений выполняется на индивидуальном сквозном проекте обучающегося с учётом его предметной области и реализованного функционала.

| № | Показатель | Ожидаемый ответ (результат) |
|---|---|---|
| 1 | Проектирование структуры базы данных | Покажите структуру базы данных своего проекта. Объясните назначение основных таблиц, состав хранимых данных и связи между сущностями предметной области. |
| 2 | Создание таблиц и связей | Покажите SQL-команды создания таблиц. Объясните назначение первичных и внешних ключей и продемонстрируйте, как они обеспечивают связь записей между таблицами. |
| 3 | Создание базы данных и выполнение схемы | Продемонстрируйте создание базы SQLite и выполнение скрипта создания таблиц. Объясните, как организовать повторный запуск скрипта, чтобы избежать ошибок при наличии существующих таблиц. |
| 4 | Подключение к базе данных | Продемонстрируйте подключение Python к файлу SQLite и выполнение простого SQL-запроса. Объясните, как убедиться, что приложение использует нужную базу данных. |
| 5 | Импорт данных из JSON | Продемонстрируйте перенос данных из JSON-файлов в таблицы SQLite. Объясните, как преобразуются исходные данные и почему при наличии связей между записями важен порядок импорта. |
| 6 | Проверка результатов импорта | Выполните SQL-запросы для подсчёта записей в таблицах. Сравните результаты с исходными JSON-файлами и объясните, как выявить неполный импорт данных. |
| 7 | Выборка данных с помощью SELECT | Покажите SQL-запрос для получения списка объектов основной сущности проекта. Измените состав выбираемых полей и объясните, как это влияет на результат запроса и данные, доступные для отображения. |
| 8 | Фильтрация данных с помощью WHERE | Покажите SQL-запрос для поиска записей по заданному условию. Измените условие отбора и продемонстрируйте, как оно влияет на состав результата. |
| 9 | Фильтрация записей по состоянию или признаку | Покажите SQL-запрос для получения записей, соответствующих определённому состоянию или признаку предметной области. Измените условие и объясните, какие записи включаются в результат и какие исключаются. |
| 10 | Объединение таблиц с помощью JOIN | Покажите SQL-запрос, объединяющий данные из нескольких связанных таблиц. Объясните условия соединения и продемонстрируйте, как запрос позволяет получить сведения о связанных сущностях. |
| 11 | Отбор данных по нескольким условиям | Покажите SQL-запрос, который выбирает записи с учётом нескольких условий предметной области, включая при необходимости дату или другой параметр. Измените одно из условий и объясните, как изменится результат. |
| 12 | Подсчёт записей с помощью COUNT() | Покажите SQL-запрос для подсчёта количества записей, соответствующих задаче проекта. Объясните назначение COUNT() и результат запроса при отсутствии подходящих записей. |
| 13 | Группировка данных с помощью GROUP BY | Покажите SQL-запрос для подсчёта или обобщения данных по категориям либо связанным сущностям. Объясните назначение GROUP BY и то, как формируются отдельные группы результатов. |
| 14 | Фильтрация групп с помощью HAVING | Покажите SQL-запрос, который отбирает группы по результатам агрегатных вычислений. Измените пороговое значение и объясните отличие HAVING от WHERE. |
| 15 | Параметризованные SQL-запросы | Покажите SQL-запрос, условие которого зависит от значения, полученного от пользователя или переданного приложением. Продемонстрируйте передачу значения отдельным параметром и объясните, почему не следует подставлять пользовательский ввод непосредственно в строку SQL. |

| № | Показатель | Ожидаемый ответ (результат) |
|---|---|---|
| 16 | Получение и обработка результатов SQL | Продемонстрируйте выполнение SQL-запроса из Python-кода Django и обработку полученных строк. Покажите обращение к полям результата по именам столбцов и объясните назначение соответствующей настройки SQLite. |
| 17 | Отображение списка объектов | Продемонстрируйте страницу со списком объектов основной сущности проекта, данные для которой получены из SQLite. Объясните, как результат SQL-запроса передаётся из представления в шаблон. |
| 18 | Отображение отфильтрованных записей | Продемонстрируйте страницу, отображающую записи, отобранные по условию предметной области. Объясните, как SQL-фильтрация определяет состав отображаемых данных. |
| 19 | Отображение связанных данных | Продемонстрируйте страницу, на которой одновременно представлены сведения о нескольких связанных сущностях. Объясните, как результат JOIN используется при формировании страницы. |
| 20 | Реализация поиска по пользовательским параметрам | Продемонстрируйте страницу поиска или фильтрации данных по значению, введённому или выбранному пользователем. Покажите передачу параметра в представление и выполнение соответствующего SQL-запроса. |
| 21 | Отображение статистических данных | Продемонстрируйте страницу со статистикой по данным проекта. Объясните, как результаты агрегатных запросов передаются в шаблон и отображаются пользователю. |
| 22 | Передача результатов в шаблон | Продемонстрируйте исправление ситуации, когда SQL-запрос возвращает данные, но они не отображаются из-за несоответствия имён переменных в контексте представления и шаблоне. |
| 23 | Обработка пустых результатов | Продемонстрируйте поведение страницы, если по заданным условиям не найдено ни одной записи. Объясните, чем пустой результат запроса отличается от ошибки выполнения SQL. |
| 24 | Диагностика ошибок SQL-запросов | Разберите пример некорректного результата, вызванного ошибкой в имени поля, условии отбора или соединении таблиц. Исправьте запрос и продемонстрируйте корректный результат в приложении. |
| 25 | Сквозная обработка запроса в Django | Проследите полный путь обработки пользовательского запроса: получение параметров, выполнение SQL, обработка результатов в Django и отображение данных в браузере. Объясните, на каких этапах следует искать причину некорректного результата. |
| | **Итого** | |

## 8. КОНТРОЛЬНЫЕ ВОПРОСЫ

При подготовке к защите необходимо уметь ответить на контрольные вопросы и продемонстрировать соответствующий участок проекта.

Базы данных и реляционная модель

1. Что такое база данных?

2. Чем база данных отличается от СУБД?

3. Что такое реляционная база данных?

4. Что такое таблица, запись и поле?

5. Какие типы данных поддерживает SQLite?

6. Для чего используется первичный ключ?

7. Для чего используется внешний ключ?

Выборка и фильтрация данных

8. Что такое SQL?

9. Чем SQL-запрос отличается от Python-кода?

10. Для чего используются SELECT и FROM?

11. Для чего используется WHERE?

12. Чем отличаются операторы =, <> и !=?

13. Для чего используются IN, BETWEEN и LIKE?

14. Как используются AND, OR и NOT?

15. Как в SQL проверяется значение NULL?

16. Для чего используется DISTINCT?

Сортировка, агрегация и группировка

17. Для чего используется ORDER BY?

18. Чем отличаются ASC и DESC?

19. Для чего используются LIMIT и OFFSET?

20. Почему LIMIT и OFFSET рекомендуется использовать вместе с ORDER BY?

21. Что такое агрегатная функция?

22. Чем отличаются COUNT(*) и COUNT(column)?

23. Для чего используются MIN, MAX, AVG и SUM?

24. Для чего используется GROUP BY?

25. Чем WHERE отличается от HAVING?

26. В каком порядке выполняются основные части SQL-запроса?

Работа с SQLite из Python

27. Для чего используется модуль sqlite3?

28. Какие основные этапы работы с SQLite из Python?

29. Для чего используется cursor?

30. Для чего используется commit()?

31. Почему нельзя подставлять значения пользователей в SQL через f-строки?

32. Что такое параметризованный SQL-запрос?

33. Для чего используется executemany()?

34. Чем файловая SQLite-база отличается от базы :memory:?

35. В каких случаях база :memory: удобна?

36. Как получить результаты SELECT в Python?

Индивидуальный проект

37. Какие операции с данными уже выполнялись в проекте до перехода на базу данных?

38. Какие преимущества даёт хранение данных в базе по сравнению с JSON-файлами?

39. Какие таблицы и связи используются в собственной предметной области?

40. Какие SQL-запросы необходимы для решения основных задач собственного проекта?

## 9. ИТОГ

После выполнения ПР7 индивидуальный проект получает полноценный слой реляционного хранения данных:

Минимальный объём SQL-части зафиксирован методичкой: не менее 10 SQL-запросов, в том числе не менее двух запросов каждого вида (выборка списков, фильтрация WHERE, соединение таблиц JOIN, агрегатные функции, группировка GROUP BY), – с обязательным переносом результатов запросов в веб-приложение.

```
Python → sqlite3 → SQL → SQLite → результирующая выборка
```

Спроектированы таблицы базы данных предметной области проекта (для Roomly – rooms, users, bookings), определены первичные и внешние ключи, данные перенесены из JSON-файлов в SQLite, разработаны SQL-запросы, реализующие функциональность проекта, результаты запросов получаются в Django-представлениях и отображаются на веб-страницах. JSON-хранилище ПР1–ПР6 сохраняется как предыдущая реализация предметной области.

На данном этапе Django ORM не используется: переход от SQL к моделям Django и ORM выполняется в ПР9.

Таким образом, ПР7 завершает работу с самостоятельным SQL-слоем и подготавливает индивидуальный проект к дальнейшему переходу (рис. 5):

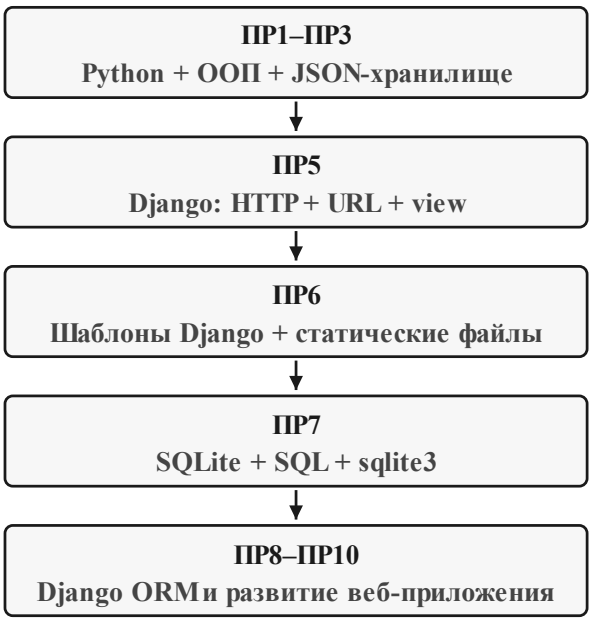



Рисунок 5 – Развитие индивидуального проекта